Observatorio de Turismo Sostenible — Guatemala

Laboratorio 11: pipeline MLOps reproducible. Informe técnico final

Noviembre de 2026

Índice

Resumen **2**

1. Introducción y alcance del observatorio **2**

2. Variable objetivo, unidad, fuente y granularidad **2**

3. Arquitectura y diagrama **3**

4. Pipeline reproducible **4**

5. Seis controles automáticos de calidad **4**

6. Procedimiento de estimación **5**

7. Evaluación temporal y baseline **6**

8. Métricas MAE y RMSE **6**

9. Regla de aceptación **7**

10. Registro y trazabilidad mediante MLflow **7**

11. Reproducibilidad de ejecuciones **8**

12. Publicación controlada y versionado **9**

13. Restauración de versiones **9**

14. Tablero Streamlit **9**

15. Monitoreo y umbrales **10**

16. Prueba de fallo controlado y recuperación **11**

17. Docker Compose y persistencia **12**

18. Limitaciones y conclusiones **13**
18.1 Limitaciones 13
18.2 Conclusiones 13

Anexo A. Evidencias y su origen **14**

Resumen

Este informe describe el pipeline MLOps del Observatorio de Turismo Sostenible de Guatemala: un flujo reproducible que lee una fuente mensual, valida su calidad con seis controles automáticos, estima el ingreso mensual de visitantes no residentes con un procedimiento estacional, lo evalúa en el tiempo contra una referencia sencilla, decide con una regla fija si el resultado puede publicarse, lo registra en MLflow, lo publica en versiones inmutables, lo muestra en un tablero Streamlit, lo monitorea y lo empaqueta en Docker Compose con almacenamiento persistente.

Advertencia sobre las cifras. Todas las métricas y valores numéricos de resultados que aparecen en este informe (MAE 42.75, RMSE 50.892861319966414, etc.) se obtuvieron con **datos sintéticos** generados por `scripts/make_synthetic_data.py`. Sirven para demostrar y probar el flujo; **no son resultados institucionales** y no permiten concluir nada sobre el desempeño del procedimiento con la fuente real. Las secciones indican siempre qué evidencia se revisó en el repositorio y cuál fue reportada por el equipo (Anexo A).

1. Introducción y alcance del observatorio

El observatorio responde a una pregunta operativa: ¿cuántos visitantes no residentes ingresan a Guatemala cada mes y cuántos se esperan en los próximos meses? El laboratorio no se centra en el modelo, sino en el **ciclo de vida** que lo rodea: que cada resultado sea reproducible, validado, trazable, publicable de forma controlada, recuperable ante fallos y consultable por directivos sin ejecutar código.

Alcance implementado:

Componente	Módulo o archivo principal
Lectura, controles de calidad y preparación	`pipeline/io.py`, `checks.py`, `dates.py`, `prepare.py`, `report.py`
Estimación, evaluación temporal y aceptación	`pipeline/evaluation.py`
Contrato de resultados y metadatos	`pipeline/contract.py`, `metadata.py`
Registro en MLflow	`pipeline/tracking.py`
Monitoreo y alertas	`pipeline/monitoring.py`
Publicación y restauración	`pipeline/publish.py`, `restore.py`
Tablero	`app.py`, `dashboard/`
Entorno reproducible	`Dockerfile`, `docker-compose.yml`, `scripts/docker_build.ps1`
Pruebas	`tests/` (ocho módulos)

Fuera de alcance: modelos de aprendizaje profundo (el procedimiento es deliberadamente simple), desgloses por vía, región o residencia en el tablero, autenticación, notificaciones externas y programación periódica de ejecuciones. El tablero y MLflow están pensados para uso local.

2. Variable objetivo, unidad, fuente y granularidad

Elemento	Definición
Variable	Ingreso mensual de visitantes no residentes: Turista + Excursionista (`ingreso_mensual_visitantes_no_residentes`)

Elemento	Definición
Unidad	Visitantes por mes
Destino	Guatemala, total nacional. Es el destino de trabajo: no se afirma que sea el destino asignado oficialmente
Granularidad	Mensual: una cifra por mes y destino
Fuente institucional	`Base_Migracion_2009-2026jun.xlsx`, hoja `Datos` (la misma del Laboratorio 2). Se monta desde fuera del repositorio y de la imagen Docker
Fuente sintética	`data/synthetic/base_sintetica.csv`, generada con semilla 42 para enero de 2009 a junio de 2026

Según `docs/ALCANCE.md`, la fuente institucional tiene 161,036 filas y 13 columnas, cubre 210 meses consecutivos (2009-01 a 2026-06) y su población de referencia (Turista y Excursionista) reúne 137,642 filas. «Viajero» (cruce fronterizo sin ser visitante) y «Cruceristas» (solo hasta 2022) se excluyen con un filtro configurable. La granularidad real de la fuente es mes × vía × frontera × país × agrupación de residencia × tipo de viajero; la serie mensual se obtiene **sumando** las filas de la población de referencia de cada mes, no presencias diarias. Esas cifras de la fuente real provienen de la documentación del avance y no se reprodujeron para este informe.

La muestra sintética que respalda toda la evidencia numérica de este informe tiene 22,848 filas (`metadata.json` de la ejecución `20261104T232908Z-48fea1`) y la huella SHA-256 `27f79fa77eae5751f46e15e3e11d807b50241` registrada en `config/config.yaml` como sintética.

Clasificación de la fuente. Cada ejecución clasifica su fuente como `sintetica`, `institucional` o `no_declarada` (`pipeline.metadata.classify_source`) sin usar el nombre del archivo: primero la huella SHA-256 conocida, luego la declaración `--source-kind`, luego `source.kind` de la configuración solo si no se reemplazó la ruta con `--data`. Una fuente `no_declarada` nunca se publica, y lo sintético nunca se presenta como institucional.

Limitaciones de la fuente real (hoja `Notas`, según la documentación): uso exclusivamente académico; quiebre metodológico entre 2022 y 2023; cambio de granularidad de `País` desde 2023; pérdida de detalle de la vía marítima desde 2017; colapso por la pandemia en 2020 y 2021. Por el quiebre, el ajuste se declara desde 2023-01 y la evaluación en 2025.

3. Arquitectura y diagrama

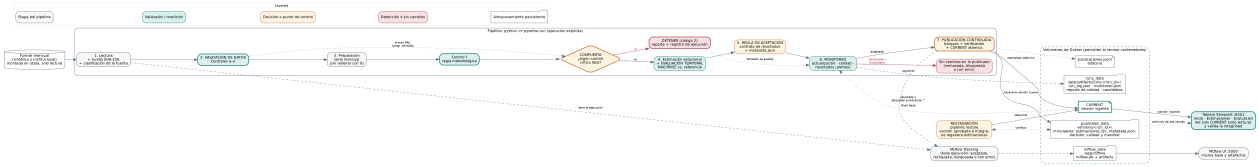


Figura 1: Diagrama de flujo del observatorio (fuente: `docs/flujo.dot`)

El diagrama (`docs/diagrama_flujo.png` y `.svg`, generados desde `docs/flujo.dot`) muestra los siete pasos del pipeline, los almacenes persistentes y los consumidores. La arquitectura se organiza en tres capas:

1. **Pipeline** (`python -m pipeline.run`): se ejecuta de forma explícita, produce una carpeta por ejecución y, si el resultado es aceptado, lo publica.
2. **Almacenamiento**: carpeta de ejecuciones (`artifacts/runs/<run_id>/`), almacén de MLflow (`mlflow.db` y artefactos) y área de publicación (`published/versions/<run_id>/`, `CURRENT`, `publicaciones.jsonl`).
3. **Consumo**: el tablero Streamlit lee únicamente la versión que señala `CURRENT`; MLflow UI lee el almacén de MLflow.

Principios de diseño: (a) lo publicado solo cambia mediante el módulo de publicación o de restauración; (b) un fallo nunca modifica lo publicado; (c) cada resultado conserva el identificador de su ejecución, que es también el identificador de la versión; (d) ningún dato institucional entra en el repositorio, en las imágenes ni en los resultados publicados.

4. Pipeline reproducible

El comando `python -m pipeline.run --config config/config.yaml [--data RUTA] [--source-kind sintetica|institucional]` ejecuta las siguientes etapas (`pipeline/run.py`), cada una con hora de inicio, fin y estado en el registro de la ejecución:

Etapa	Qué hace
`lectura`	Lee la fuente, calcula SHA-256, tamaño y filas, y clasifica el tipo de fuente
`controles_a_e`	Aplica el filtro de población y los controles a) a e)
`preparacion`	Construye la serie mensual sin rellenar con cero; se omite si algún control crítico falló
`control_f`	Verifica la regla metodológica
`compuerta_calidad`	Escribe el reporte de calidad; si algún control falló, termina con código 2
`estimacion_y_evaluacion`, `estimaciones_candidatas`	Evalúa en el tiempo y genera las estimaciones candidatas
`regla_aceptacion`, `metadatos`	Decide la aceptación y escribe `metadata.json`
`monitoreo`	Evalúa alertas de actualización, calidad y resultados
`publicacion`	Publica la versión si fue aceptada
`registro_mlflow`	Cierra la ejecución en MLflow (siempre, incluso con errores)

Códigos de salida: 0 = completado (aceptado o rechazado por la regla, con o sin advertencias); 2 = control crítico fallido; 1 = error inesperado, incluido un fallo técnico al publicar.

Configuración. Todo lo que cambia entre ejecuciones vive en `config/config.yaml`: rutas, períodos (esperado 2009-01 a 2026-06, ajuste 2023-01 a 2024-12, evaluación 2025-01 a 2025-12, horizonte 2026-07 a 2026-12), parámetros del estimador, umbrales de calidad, regla de aceptación, monitoreo, publicación y MLflow. La ejecución guarda una copia (`config_snapshot.yaml`) y su huella SHA-256.

Artefactos de cada ejecución (`artifacts/runs/<run_id>/`): `quality_report.json` y `.md`, `serie_mensual_total.csv` y `serie_mensual_por_dimension.csv` (solo si no hubo fallos críticos), `evaluacion_temporal.csv`, `estimaciones_candidatas.csv`, `decision_aceptacion.json`, `metadata.json`, `monitoreo.json`, `config_snapshot.yaml` y `run_log.json`.

Registro por ejecución (`run_log.json`): hora de inicio y fin, duración, etapas con su estado, `etapas_completadas`, advertencias, errores, resultado de publicación, `version_publicada`, resumen de MLflow, alertas de monitoreo, y commit de Git con su origen. El `run_id` tiene la forma `AAAAMMDDTHHMMSSZ-xxxxxx`.

5. Seis controles automáticos de calidad

Cada control devuelve PASS, WARN (se reporta y el flujo continúa) o FAIL (el pipeline se detiene antes de publicar y lo publicado no cambia). Se aplican a la población de referencia. Los umbrales están en `config/config.yaml`, sección `quality`.

Control	Qué verifica	Advertencia	Detiene
a) Columnas y tipos	Columnas indispensables con el tipo esperado	Falta una columna opcional	Falta una requerida o hay valores no convertibles (tolerancia 0)
b) Fechas y cobertura	Fechas válidas; cada mes de ajuste y evaluación tiene datos	Fechas inválidas bajo 0.1 %, meses sin datos fuera de los períodos requeridos, fechas fuera de rango	Fechas inválidas sobre 0.1 % o meses requeridos sin datos
c) Unicidad	Sin duplicados para la clave mes + vía + frontera + país + agrupación + tipo de viajero	Duplicados bajo el umbral (hoy 0)	Cualquier duplicado
d) Valores faltantes	Distingue faltantes permitidos (dimensiones, etiquetados «Sin dato») de los que impiden estimar	Faltantes permitidos hasta 5 %	Cualquier faltante en la variable a estimar, o permitidos sobre 5 %
e) Valores y denominadores	Sin negativos, sin infinitos, total mensual distinto de cero	Total cero si la política es `warn`	Negativos, infinitos o total mensual cero (política `fail`)
f) Regla metodológica	La serie viene de la fuente mensual, cuadra con ella y no rellena con cero meses sin cobertura	No aplica	Fuente diaria, serie que no cuadra (tolerancia relativa 1e-9) o mes sin cobertura con valor

Si el control a) falla, los demás se marcan NO EVALUADO. Si cualquier control falla, la preparación se omite y no se escribe ninguna serie.

Evidencia. En la ejecución sintética `20261104T232908Z-48fea1`, `quality_report.json` registra estado global PASS y los seis controles en PASS (revisado en el repositorio). El equipo reportó seis PASS en todas las ejecuciones válidas. Las pruebas inyectan siete fallas con `scripts/make_synthetic_data.py` (`missing_column`, `duplicate_key`, `zero_total`, `negative_value`, `missing_month`, `invalid_date`, `daily_dates_in_value_text`) y verifican que cada una termina con código 2 sin tocar `published/` (`tests/test_pipeline.py`).

6. Procedimiento de estimación

El procedimiento es una **referencia estacional ingenua** (`seasonal_naive`): la estimación de un mes es el valor observado del mismo mes del año anterior,

```
y_estimado(t) = y_observado(t - 12 meses)
```

con período estacional 12 (`parameters.seasonal_period`). Es determinista, no tiene parámetros que ajustar, no reentrena nada y conserva el patrón estacional de una serie mensual. La semilla (42) se registra por trazabilidad, pero el estimador no usa aleatoriedad.

La serie sobre la que se estima proviene de `pipeline/prepare.py`: suma mensual de la variable `Viajero` para la población de referencia, con una marca de cobertura. Un mes sin cobertura queda vacío (nunca cero) y no se usa como referencia: si faltara la observación de hace 12 meses, la estimación de ese mes no se produce y el candidato no cumple el contrato de horizonte completo.

El horizonte entregado es julio a diciembre de 2026 (6 meses). Ejemplo con la ejecución sintética `20261104T232908Z-48fea1`: la estimación de julio de 2026 es 2,170 visitantes por mes, igual a la observación sintética de julio de 2025 (2,170).

La ventana de ajuste (2023-01 a 2024-12) solo define la cobertura exigida por los controles; el estimador no ajusta parámetros con ella. Esta limitación está declarada en `metadata.json` (`ventana_de_ajuste_sin_parametros`).

Contrato de resultados. Las estimaciones candidatas (`estimaciones_candidatas.csv`) tienen una fila por destino y mes con `destino, periodo, tipo (observado|estimado), valor, unidad, run_id, fecha_generacion, version_procedimiento, metodo`. `pipeline/contract.py` valida columnas, valores finitos y no negativos, períodos mensuales sin duplicados, consistencia de columnas constantes y horizonte exacto. Un candidato que no cumple el contrato no puede ser aceptado. La ejecución `20261104T232908Z-48fea1` tiene 216 filas (210 meses observados y 6 estimados) y contrato válido.

7. Evaluación temporal y baseline

La evaluación usa un **origen móvil** sobre enero a diciembre de 2025. Para cada mes: (1) se descartan conceptualmente su observación y todas las posteriores; (2) se construye la estimación solo con datos anteriores; (3) se compara con el valor observado oculto. La prueba `test_temporal_evaluation_does_not_use_the_target_month` cambia la observación del mes evaluado y comprueba que ni su estimación ni su referencia cambian.

La **referencia (baseline)** es el último valor observado antes del mes evaluado, calculada con la misma información disponible. Un mes sin observación comparable se marca NO EVALUADO y no genera métrica ni se convierte en cero.

Detalle de la ejecución sintética `20261104T232908Z-48fea1` (`evaluacion_temporal.csv`; valores en visitantes por mes, **datos sintéticos**):

Mes	Observado	Estimación	Referencia	Error abs. estimación	Error abs. referencia
2025-01	1,667	1,681	1,532	14	135
2025-02	1,917	1,871	1,667	46	250
2025-03	2,193	2,101	1,917	92	276
2025-04	2,316	2,337	2,193	21	123
2025-05	2,430	2,398	2,316	32	114
2025-06	2,321	2,336	2,430	15	109
2025-07	2,170	2,096	2,321	74	151
2025-08	1,859	1,879	2,170	20	311
2025-09	1,652	1,695	1,859	43	207
2025-10	1,508	1,476	1,652	32	144
2025-11	1,466	1,371	1,508	95	42
2025-12	1,503	1,532	1,466	29	37

El procedimiento supera a la referencia en 11 de los 12 meses; en noviembre la referencia es mejor. Este resultado depende de que la muestra sintética está construida con una estacionalidad regular, que el procedimiento estacional aprovecha por diseño.

Limitación. La fuente es una fotografía final: el mecanismo simula el corte temporal pero no reproduce exactamente la información disponible en cada fecha de corte real. La evaluación es una sola ventana de 12 meses.

8. Métricas MAE y RMSE

Con `e_i` el error de la estimación en el mes evaluado `i` y `n` los meses evaluados:

```
MAE  = (1/n) * suma(|e_i|)
RMSE = raiz( (1/n) * suma(e_i^2) )
```

El MAE es el error promedio en visitantes por mes; el RMSE penaliza más los errores grandes y es siempre mayor o igual que el MAE. Ambos se calculan para el procedimiento y para la referencia.

Resultados con **datos sintéticos** (12 meses evaluados, 2025-01 a 2025-12; `decision_aceptacion.json`):

Métrica	Procedimiento	Referencia
MAE (visitantes por mes)	42.75	158.25
RMSE (visitantes por mes)	50.892861319966414	178.6866437836546

Verificación independiente con la tabla de la sección 7: la suma de errores absolutos del procedimiento es 513 (513 / 12 = 42.75) y la de la referencia es 1,899 (1,899 / 12 = 158.25); la suma de errores al cuadrado del procedimiento es 31,081 (raíz de 31,081 / 12 = 50.8929). La razón entre los MAE es 42.75 / 158.25, aproximadamente 0.270, y entre los RMSE aproximadamente 0.285.

Estas cifras describen únicamente la muestra sintética. No son una evaluación institucional del procedimiento.

9. Regla de aceptación

La regla es fija (`config/config.yaml`, sección `acceptance`) y no se ajusta después de ver una ejecución. Un candidato queda **aceptado** solo si se cumplen las cuatro verificaciones:

Verificación	Condición
`calidad_sin_fallos_criticos`	Ningún control de calidad tiene estado FAIL
`salidas_completas`	Las estimaciones cumplen el contrato de resultados
`meses_evaluados_suficientes`	Al menos 6 meses evaluados (`min_evaluated_months: 6`)
`mae_dentro_del_umbral`	MAE del procedimiento menor o igual a 1.10 veces el MAE de la referencia (`max_mae_ratio_to_baseline: 1.10`)

La decisión y sus verificaciones se guardan en `decision_aceptacion.json`. Una decisión aceptada solo **autoriza** la publicación: no la realiza. Con los datos sintéticos, el umbral del MAE es 1.10 × 158.25 = 174.075; el MAE del procedimiento (42.75) lo cumple y las cuatro verificaciones son verdaderas, sin motivos de rechazo (`aceptada: true`, revisado en `decision_aceptacion.json`). Las pruebas cubren el caso de rechazo (por ejemplo, exigir 99 meses evaluados) y verifican que un rechazado no modifica `published/`. Rechazar un candidato no es un error técnico: la ejecución termina con código 0.

10. Registro y trazabilidad mediante MLflow

Cada ejecución, aceptada, rechazada, detenida por calidad o con error, se registra con MLflow Tracking en un almacén local: base SQLite (`mlflow.db`) y carpeta de artefactos, dentro del experimento `observatorio_turismo_guatemala` (`pipeline/tracking.py`). El `run_id` del pipeline y el identificador de MLflow son distintos y se relacionan con la etiqueta `pipeline.run_id`; el identificador de MLflow queda en `run_log.json` y en `metadata.json`.

Tipo	Contenido
Parámetros	Destino, variable, unidad, períodos, método, período estacional, baseline, semilla y umbrales de aceptación
Etiquetas	`pipeline.run_id`, versión, commit de Git y su origen (`git.commit`, `git.commit_origen`, `git.tree_modified`), huella de la configuración, tipo de fuente y su huella, resultado, decisión y motivos, estado de cada control (`calidad.<id>`), `publicacion.*`, `monitoreo.*`, `trazabilidad`
Métricas	`mae_estimacion`, `rmse_estimacion`, `mae_referencia`, `rmse_referencia`, `razon_mae`, meses evaluados, controles por estado y `monitoreo.alertas_*`
Artefactos	Reporte de calidad, evaluación temporal, estimaciones candidatas, decisión, metadatos, monitoreo y copia de la configuración

No se registran el archivo fuente, las series mensuales ni `run_log.json`, y los textos de error pasan por una función que reemplaza rutas absolutas por el nombre del archivo.

El estado técnico de MLflow es independiente de la decisión: FINISHED para aceptadas, rechazadas y detenidas por calidad; FAILED para error inesperado; KILLED si se interrumpe. La etiqueta `trazabilidad` indica si el registro quedó `completa`, `incompleta`, `no_disponible` o `deshabilitada`; un fallo de MLflow no cambia el resultado ni el código de salida del pipeline.

Evidencia: el equipo reportó que MLflow conserva las ejecuciones al recrear los contenedores. En el repositorio, `metadata.json` y `run_log.json` de cada ejecución incluyen el resumen de MLflow, y `tests/test_tracking.py` verifica etiquetas, métricas, artefactos y estados con un almacén temporal.

11. Reproducibilidad de ejecuciones

La reproducibilidad se apoya en cuatro elementos: configuración versionada y su huella SHA-256 en cada ejecución; huella SHA-256 de la fuente; estimador determinista (sin aleatoriedad); y registro del commit de Git del código.

Evidencia. El equipo confirmó que dos ejecuciones sintéticas con la misma fuente y configuración produjeron las mismas métricas. En el repositorio local se revisaron nueve carpetas de ejecución (`artifacts/runs/`): los nueve `decision_aceptacion.json` contienen exactamente MAE 42.75, RMSE 50.892861319966414, MAE de referencia 158.25, RMSE de referencia 178.6866437836546 y 12 meses evaluados. Además, `tests/test_tracking.py` compara dos ejecuciones y exige huellas, resultado, parámetros y métricas idénticos.

Lo que difiere entre ejecuciones es esperable: el `run_id`, la fecha de generación y los identificadores de MLflow.

Commit de Git. Fuera de Docker, el pipeline lee el commit con Git. Dentro de la imagen no hay `.git`: el commit se declara al construir (argumentos `GIT_COMMIT` y `GIT_TREE_MODIFIED`, variables `LAB11_GIT_COMMIT` y `LAB11_GIT_TREE_MODIFIED`) y se registra con su origen (`git`, `variable_de_entorno` o `no_disponible`) en `run_log.json`, `metadata.json` y MLflow. El equipo reportó que una ejecución en Docker registró el commit `a96775c92fa7028d5cbbc1d3aff7c27c67c2a5f3`. Es una declaración de quien construye la imagen, no una verificación, y solo identifica el código si `arbol_git_modificado` es falso.

Límite. Las dependencias de `requirements.txt` no están todas fijadas (por ejemplo `mlflow>=2.14`): una imagen reconstruida meses después podría resolver versiones distintas.

12. Publicación controlada y versionado

Una ejecución aceptada se publica como **versión inmutable** en `published/versions/<run_id>/`; el identificador de la versión es el `run_id` del pipeline. Cada versión contiene `estimaciones.csv`, `metadata.json` (con el bloque `publicacion`: versión, fecha y versión anterior), `decision_aceptacion.json`, `quality_report.json` y `manifest.json` con huella SHA-256 y tamaño de los otros cuatro archivos. El archivo `published/CURRENT` contiene el identificador de la versión vigente y `publicaciones.jsonl` registra cada evento.

Condiciones (función `check_bundle`, compartida con la verificación y la restauración): decisión favorable; contrato válido, validado con los metadatos de la propia versión; seis controles sin fallo ni «no evaluado»; metadatos válidos y sin rutas absolutas; identificadores coherentes; huella de las estimaciones igual a la de los metadatos; fuente `sintetica` o `institucional` (nunca `no_declarada`); y que la versión no exista.

Actualización atómica. (1) Bloqueo exclusivo del sistema operativo (`msvcrt` en Windows, `flock` en Linux) para serializar publicaciones simultáneas. (2) La versión se prepara completa en `.staging/`. (3) Se verifica con la misma función que usa la restauración. (4) Se mueve a `versions/` con un renombrado que no sobrescribe. (5) `CURRENT` se reemplaza con un archivo temporal, `fsync` y `os.replace`. (6) Se registra el evento. Si falla la actualización de `CURRENT`, la versión recién movida se retira y la vigente se conserva. Se conservan todas las versiones aprobadas.

Publicación sintética. Las versiones sintéticas se pueden publicar localmente (`publication.allow_synthetic`), identificadas como tales en sus metadatos, en el manifest y en la bitácora; el tablero las rotula. Una versión sintética vigente reemplaza a una institucional en `CURRENT` como cualquier otra: la identificación visible es la protección.

Evidencia. La ejecución `20261104T232908Z-48fea1` terminó con código 0, `resultado_publicacion: publicada` y `version_publicada: 20261104T232908Z-48fea1` (`run_log.json`). `tests/test_publish.py` cubre publicaciones sucesivas, rechazos, corrupción de cuatro tipos, fallos al actualizar `CURRENT`, concurrencia y la integración con el pipeline.

13. Restauración de versiones

El comando `python -m pipeline.restore --list` muestra la versión vigente y la validez de cada versión; `--version <run_id> --reason "motivo"` vuelve a señalar con `CURRENT` una versión anterior. La restauración:

- exige un identificador válido y una versión existente, íntegra (huellas del manifest, sin archivos faltantes ni ajenos) y aprobada;
- **no regenera estimaciones** ni modifica versiones;
- usa el mismo bloqueo y reemplazo atómico que la publicación;
- registra en `publicaciones.jsonl` la versión anterior y el motivo;
- rechaza versiones inexistentes, corruptas o no aprobadas sin cambiar `CURRENT`.

Códigos de salida: 0 restaurada, ya vigente o listado; 2 rechazada; 1 error técnico.

Evidencia. El equipo reportó la restauración verificada tanto en la prueba de recuperación como en Docker. Las pruebas automáticas cubren la restauración de la versión anterior, de versiones inexistentes o con identificador inválido, de versiones dañadas y de versiones no aprobadas (`tests/test_publish.py`), y el ciclo completo de fallo y restauración (`tests/test_monitoring.py`).

14. Tablero Streamlit

El tablero (`app.py`, `dashboard/`) tiene tres páginas y está orientado a directivos:

Página	Contenido
Inicio	Nombre del observatorio, destino, variable y unidad, último período y valor observados, fecha de publicación, versión y tipo de fuente, y resumen de las estimaciones publicadas
Estimaciones	Serie temporal con observados (gris oscuro) y estimados (turquesa discontinuo), horizonte estimado sombreado y selector de período
Evaluación	Decisión de aceptación y su regla, MAE y RMSE del procedimiento y de la referencia, meses y período evaluados, explicación de las métricas y limitaciones metodológicas desde `metadata.json`

Lectura de versiones aprobadas. `dashboard/published.py` lee `published/CURRENT`, abre únicamente `versions/<run_id>/`, y la valida con las funciones del módulo de publicación (`verify_directory`) antes de mostrar cifras. No ejecuta el pipeline, no lee el Excel institucional, no importa módulos de modelos y no usa los resultados candidatos. Cambiar filtros solo acota lo dibujado.

Estados sin cifras. Si `CURRENT` no existe, muestra «Aún no hay resultados aprobados» sin datos de ejemplo; si la versión vigente está dañada, no está aprobada o tiene metadatos incompletos, muestra un error comprensible sin rutas locales. En ambos casos la navegación sigue funcionando.

Fuente sintética. Si `fuente.tipo` es `sintetica`, todas las páginas muestran un aviso ámbar, las series se rotulan «sintético» y la página de evaluación advierte que las métricas no son una evaluación institucional.

El diseño visual (marca turquesa, tipografía, tarjetas y formatos) se adaptó del tablero del Laboratorio 10; no se copiaron sus cargadores, modelos LSTM ni perfiles de series.

Evidencia. El equipo reportó que Streamlit funciona y conserva las estimaciones al recrear los contenedores. `tests/test_dashboard.py` prueba el cargador (versión válida, `CURRENT` ausente o inválido, versión inexistente, corrupta o con metadatos incompletos), la ausencia de dependencias de modelos, que no se ejecuta el pipeline ni se escribe en `published/`, y las páginas con `streamlit.testing`. Este informe no incluye capturas de pantalla.

15. Monitoreo y umbrales

`pipeline/monitoring.py` evalúa señales después de la regla de aceptación y antes de la publicación, también cuando la compuerta de calidad detiene la ejecución. **Informa; no cambia códigos de salida, aceptación ni publicación.** Reutiliza los detalles de los controles de calidad y compara contra la versión vigente (`CURRENT`) como línea base, solo si es íntegra y de la misma clase de fuente. Las alertas tienen severidad `critica` (la condición ya impide publicar por otra vía) o `advertencia` (requiere revisión humana).

Familia	Alertas
Actualización	`fuente_desactualizada`, `meses_requeridos_ausentes` (crítica), `huecos_en_la_serie`, `horizonte_incompleto` (crítica), `publicacion_desactualizada`, `publicacion_antigua`, `candidato_con_menos_datos`
Calidad	`faltantes_altos`, `faltantes_aumentaron`, `duplicados_aumentaron`, `perdida_de_cobertura`

Familia	Alertas
Resultados	`estimaciones_negativas` y `valores_no_finitos` (críticas), `variacion_mensual_alta` (advertencia de revisión)

Una fuente desactualizada (el último mes observado atrasado respecto del esperado) es distinta de un horizonte incompleto (menos meses estimados de los configurados): la primera mide los datos y la segunda la salida. Ninguna alerta recalcula errores predictivos: no hay observaciones comparables para los meses estimados.

Umbral (`monitoring` en `config.yaml`)	Valor	Justificación y límite
`max_months_behind`	0	Fuente mensual con último mes esperado declarado; si ese mes se edita a mano, la alerta depende de mantenerlo actualizado
`max_publication_age_days`	45	Un mes más dos semanas de tolerancia; no se conoce el calendario real de publicación de la fuente
`missing_warn_fraction`	0.02	Avisa antes del umbral que detiene (0.05). El nivel documentado de la fuente real es de 13 filas con «0» en «Región dos» de 161,036
`missing_max_increase`	0.005	Medio punto porcentual sobre la versión vigente
`duplicates_max_increase`	0.0	La clave es exacta y el control c detiene con cualquier duplicado
`coverage_max_months_lost`	0	La cobertura publicada es mensual y sin huecos
`max_monthly_change`	0.40	**No se midió la variabilidad mensual de la fuente real.** Es un nivel de revisión inicial conservador, no derivado de los datos

Para calibrar el último umbral, la alerta incluye la mayor variación mensual observada desde el inicio de la ventana de ajuste. Una variación alta es una advertencia de revisión: puede deberse a la estacionalidad del año anterior y no demuestra pérdida de precisión.

Evidencia. En la ejecución `20261104T232908Z-48fea1`, `monitoreo.json` registra estado `sin_alertas`, 0 alertas críticas, 0 advertencias y línea base disponible. `tests/test_monitoring.py` (43 pruebas, reportadas como aprobadas por el equipo) cubre cada alerta con casos con y sin línea base, y sus registros en `run_log.json` y MLflow.

16. Prueba de fallo controlado y recuperación

Con datos sintéticos y carpetas temporales se verificó el ciclo completo (`test_controlled_failure_keeps_current_and_the_` `tests/test_monitoring.py`), y el equipo lo repitió manualmente:

Paso	Qué ocurre
1	Se publica una versión válida; `CURRENT` la señala
2	Se introduce una falla controlada: `scripts/make_synthetic_data.py --fault duplicate_key` agrega filas con claves duplicadas
3	Se ejecuta el pipeline con esa fuente
4	El control c (unicidad) falla; la ejecución termina con **código 2**; el monitoreo emite `duplicados_aumentaron` con severidad crítica
5	`CURRENT` y todo lo publicado quedan idénticos
6	El tablero sigue mostrando la versión anterior
7	Se corrige la falla (se usa la fuente sin duplicados)
8	Se ejecuta de nuevo el pipeline
9	La ejecución es aceptada, se publica una versión nueva y `CURRENT` la señala
10	Se restaura la versión anterior con `pipeline.restore`; el tablero la muestra y las dos versiones se conservan

El equipo reportó la detección del fallo por duplicados, el código de salida 2, la conservación de `CURRENT`, la recuperación y la restauración verificadas. Las pruebas del mismo módulo repiten la conservación de `CURRENT` con las fallas `zero_total`, `negative_value` y `missing_month`. El procedimiento de recuperación general está en `docs/MONITOREO.md`.

17. Docker Compose y persistencia

Una sola imagen (`python:3.11-slim`, usuario sin privilegios) se usa para tres servicios (`docker-compose.yml`):

Servicio	Función	Puerto local	Se inicia con `up`
`mlflow`	Visor de MLflow sobre la base SQLite y los artefactos compartidos	127.0.0.1:5000	sí
`streamlit`	Tablero; monta `published/` en solo lectura, sin datos ni acceso a ejecutar el pipeline	127.0.0.1:8501	sí
`pipeline`	Se ejecuta de forma explícita con `docker compose run --rm pipeline <comando>`; su comando por defecto solo muestra la ayuda	ninguno	no (perfil propio)

Persistencia. Tres volúmenes con nombre: `mlflow_data` (`/app/mlflow`: base SQLite y artefactos), `published_data` (`/app/published`: versiones, `CURRENT` y bitácora) y `runs_data` (`/app/artifacts`: registros de ejecuciones, `monitoreo.json`, reportes). `docker compose down` y la recreación de contenedores no los eliminan; solo `down -v` lo hace. Se usan volúmenes de Docker y no carpetas de Windows porque SQLite y el bloqueo de publicación no son fiables sobre carpetas de Windows montadas.

Datos. La imagen no contiene datos: el `Dockerfile` copia solo el código y la configuración, y `.dockerignore` excluye los datos, los resultados y los secretos. La fuente se monta desde `LAB11_DATA_DIR` en `/data`, solo

lectura y solo para el servicio `pipeline`. La demostración completa usa la muestra sintética del repositorio. Los puertos se publican solo en `127.0.0.1` porque MLflow y Streamlit no tienen autenticación.

Trazabilidad. `scripts/docker_build.ps1` declara a la imagen el commit de Git y si el árbol tenía cambios sin confirmar; la imagen no incluye `.git`.

Evidencia reportada por el equipo: la imagen se construye; Streamlit y MLflow funcionan; el pipeline se ejecuta dentro de Docker; las estimaciones aprobadas se publican; `CURRENT`, el tablero y MLflow conservan sus datos al recrear los contenedores; y el commit `a96775c92fa7028d5cbbc1d3aff7c27c67c2a5f3` quedó registrado en una ejecución en Docker. `tests/test_docker.py` y `tests/test_traceability.py` comprueban solo la configuración y el manejo del commit; no ejecutan Docker.

18. Limitaciones y conclusiones

18.1 Limitaciones

1. **Toda la evidencia numérica es sintética.** Las métricas de este informe describen la muestra sintética y no son un resultado institucional. En particular, que el procedimiento estacional supere a la referencia en la muestra sintética no demuestra que lo haga con la fuente real. Los resultados con el Excel institucional no forman parte de la evidencia de este informe.
2. **Procedimiento simple.** La referencia estacional ingenua no modela tendencia, quiebres ni incertidumbre, y no tiene intervalos. La ventana de ajuste no ajusta parámetros.
3. **Evaluación en una sola ventana** de 12 meses, y sobre una fotografía final de la fuente, que no reproduce la información disponible en cada fecha de corte.
4. **Quiebre metodológico 2022–2023 y pandemia** en la fuente real: limitan qué historia es comparable.
5. **Umbrales de monitoreo iniciales.** El de variación mensual (0.40) no se calibró con la serie institucional; el de antigüedad de la publicación no conoce el calendario de la fuente.
6. **Alcance del monitoreo.** Solo corre al ejecutar el pipeline, no vigila `published/` entre ejecuciones, no envía notificaciones, no se programa y no se muestra en el tablero; la línea base es la versión vigente.
7. **Seguridad.** Sin autenticación en el tablero ni en MLflow; el manifest no está firmado (quien altere un archivo y su manifest a la vez no sería detectado).
8. **Reproducibilidad del entorno.** Dependencias sin versión fija en `requirements.txt`; el commit en Docker es una declaración del constructor.
9. **Publicación sintética.** Una versión sintética vigente reemplaza a una institucional en `CURRENT`; la barrera es la identificación visible.
10. **Docker.** Verificado en Docker Desktop para Windows según el equipo; otros sistemas y el uso de carpetas montadas en lugar de volúmenes no se probaron. La clonación limpia del repositorio no forma parte de la evidencia reportada.
11. **Alcance del tablero.** Un solo destino a nivel nacional, una sola versión, sin desgloses.

18.2 Conclusiones

El laboratorio produjo un flujo completo en el que cada decisión relevante es automática, registrada y reversible. La calidad de los datos se valida antes de estimar; la aceptación de un resultado depende de una regla fija, escrita antes de ejecutar; la publicación es atómica, versionada e inmutable; los fallos no modifican lo que los usuarios ven; y cada ejecución deja huellas de su fuente, su configuración y su código. La prueba de fallo controlado mostró que una fuente defectuosa se detecta, se registra y no altera la versión publicada, y que la recuperación y la restauración no requieren regenerar estimaciones.

Con datos sintéticos, el procedimiento estacional obtuvo MAE 42.75 y RMSE 50.892861319966414 frente a 158.25 y 178.6866437836546 de la referencia en 12 meses evaluados, y fue aceptado por la regla fija; las dos ejecuciones comparadas dieron las mismas métricas. Esos números demuestran que la cadena de evaluación, aceptación y publicación funciona de punta a punta y es reproducible, no la calidad de las estimaciones sobre la fuente institucional.

Como trabajo futuro quedan: ejecutar y documentar el flujo con la fuente institucional; calibrar los umbrales

de monitoreo con su historia; fijar versiones de dependencias; incorporar un procedimiento de estimación con incertidumbre; y automatizar la programación y las notificaciones del monitoreo.

Anexo A. Evidencias y su origen

Revisadas directamente en el repositorio para este informe

Evidencia	Dónde
MAE, RMSE, referencia, 12 meses evaluados, decisión aceptada, regla (6 meses, 1.10), contrato válido con 216 filas	`artifacts/runs/20261104T232908Z-48fea1/decision_aceptacion`
Mismas cifras en nueve ejecuciones	`artifacts/runs/*/decision_aceptacion.json`
Detalle mensual de la evaluación y cálculo manual de MAE y RMSE	`artifacts/runs/20261104T232908Z-48fea1/evaluacion_temporal`
Seis controles en PASS	`artifacts/runs/20261104T232908Z-48fea1/quality_report.json`
Fuente sintética por huella, 22,848 filas	`artifacts/runs/20261104T232908Z-48fea1/metadata.json`
Monitoreo sin alertas con línea base disponible	`artifacts/runs/20261104T232908Z-48fea1/monitoreo.json`
Publicación de la versión y código de salida 0	`artifacts/runs/20261104T232908Z-48fea1/run_log.json`
Estimaciones de julio a diciembre de 2026	`artifacts/runs/20261104T232908Z-48fea1/estimaciones_candid`
Diagrama actualizado	`docs/flujo.dot`, `docs/diagrama_flujo.svg` (contiene `CURRENT`, monitoreo y restauración), `docs/diagrama_flujo.png`
Diseño, configuración y pruebas	`pipeline/`, `dashboard/`, `config/config.yaml`, `Dockerfile`, `docker-compose.yml`, `tests/`
Documentación técnica	`docs/ALCANCE.md`, `CONTRATO_RESULTADOS.md`, `EVALUACION_Y_ACEPTACION.md`, `MLFLOW.md`, `PUBLICACION.md`, `TABLERO.md`, `MONITOREO.md`, `DOCKER.md`

Reportadas por el equipo; no verificables desde los archivos del repositorio

Evidencia reportada
296 pruebas automáticas aprobadas (43 en `tests/test_monitoring.py`)
Dos ejecuciones sintéticas con la misma fuente, configuración y métricas
Fallo controlado por duplicados detectado, con código de salida 2 y `CURRENT` conservado
Recuperación y restauración verificadas
Docker Compose funcionando; persistencia de MLflow, de `CURRENT` y de los resultados tras recrear contenedores
Commit `a96775c92fa7028d5cbbc1d3aff7c27c67c2a5f3` registrado en una ejecución en Docker
Cifras de la fuente institucional (161,036 filas, 210 meses, 137,642 filas de visitantes), tomadas de la documentación del avance

Este informe no incluye capturas de pantalla ni salidas de terminal.

Anexo B. Comandos de reproducción

Desde la raíz del repositorio, en PowerShell:

```
python -m pytest -q
python -m pipeline.run --data data/synthetic/base_sintetica.csv --source-kind sintetica
```

```
python -m pipeline.restore --list
streamlit run app.py
```

Con Docker Compose (los comandos completos, incluida la verificación de persistencia y la copia limpia, están en el `README.md`):

```
powershell -ExecutionPolicy Bypass -File .\scripts\docker_build.ps1
docker compose up -d
docker compose run --rm pipeline python -m pipeline.run --data /data/synthetic/base_sintetica.csv --sou
```

El tablero queda en http://localhost:8501 y MLflow en http://localhost:5000.